



Universidade Federal da Paraíba
Centro de Energias Alternativas e Renováveis
Departamento de Engenharia Elétrica
Automação Inteligente

Redes Neurais Artificiais

Identificação de Sistemas e Controle

Prof. Dr. Juan Mauricio Villanueva

Emmanuel Sampaio Maia da Nóbrega

Laís Araújo Manguiera

Lucas Dantas de Oliveira

Maria Eusa Alves Pinto de Oliveira

Pedro Ravel Rodrigues dos Santos

João Pessoa
2024

Sumário

1	Objetivos	1
2	Introdução	1
3	Coleta de Dados	2
4	Modelagem da Planta	4
5	Desenvolvimento da Rede Neural Artificial	5
6	Ajuste no modelo de previsão	7
6.1	Criação da RNA	7
6.2	Validação do Modelo	10
7	Desenvolvimento do sistema de controle PID	12
8	Resultados	16
9	Considerações finais	18

1 Objetivos

O objetivo desse trabalho é desenvolver um sistema de controle de pressão para uma planta no laboratório LENHS/UFPB, no qual a partir dos dados coletados seja possível a modelagem da planta, o desenvolvimento e treinamento de uma Rede Neural Artificial, o projeto do controlador PID integrado ao sistema e validação desse sistema com simulações no Software Simulink/ MATLAB.

2 Introdução

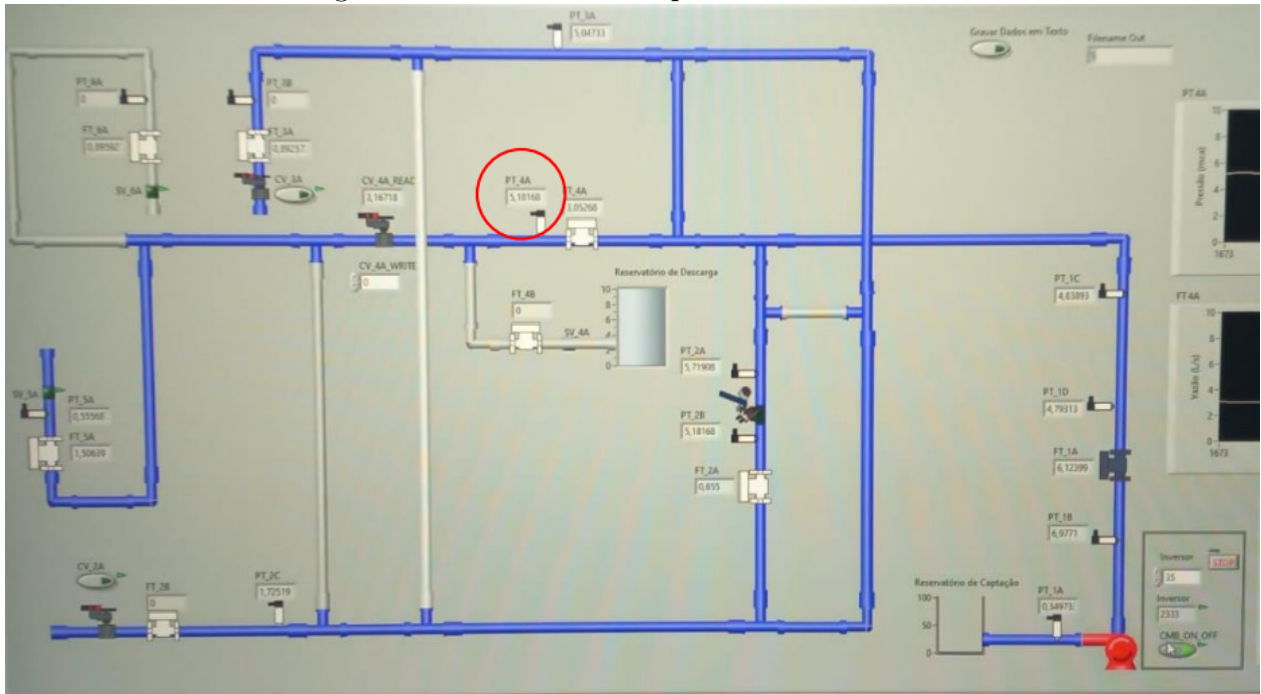
O desenvolvimento de sistemas de controle é essencial para viabilidade de operações industriais e gestão de recursos na sociedade. Dessa forma, para o cenário hídrico, o controle de variáveis como pressão e temperatura garante que o sistema de abastecimento de água ocorra de maneira segura para os usuários e com menor índice de perda para a concessionária. Isso ocorre, pois, é necessário garantir um pressão mínima para que a água chegue aos centros de distribuições e residências, mas que ao mesmo passo não seja tão elevada a ponto de romper as estruturas de distribuição e provocarem vazamentos.

Diante desse cenário de estudo, o LENHS é o laboratório de hidráulica presente na UFPB que permite, por meio da simulação de uma estação de abastecimento, o desenvolvimento de estruturas de controle para essas variáveis.

Com o avançar tecnológico da inteligência artificial, as redes neurais artificiais ganharam ainda mais espaço para aplicações em situações de análise de comportamento e previsão de demanda. Logo, essa foi uma das estratégias utilizadas para resolução do problema proposto pelo professor da disciplina.

Assim, a posposta dessa atividade é a partir de dados coletados de uma planta que simula um sistema hidráulico, realizar um modelo de previsão para variável PT_A4, conforme ilustrada na Figura 1. Para isso, foi realizado a coleta de dados, a modelagem da planta, a verificação e validação dos dados obtidos e por fim a descrição dos resultados.

Figura 1: Planta utilizada para coleta de dados.



3 Coleta de Dados

A primeira etapa do trabalho é a coleta de dados da planta, dessa forma foi solicitado as seguintes variações do inversor de frequência e do ângulo de fechamento da válvula:

Variar a frequência em: 30, 35, 40, 45, 50 Hz.

Variar o ângulo da válvula CV_4A: 0°, 10°, 20°, 30°, 40°, 50°, 60°, 70°.

Encontramos uma dificuldade ao iniciarmos a coleta de dados, pois ao variar o ângulo da válvula CV_4A acima de 30° percebíamos que esse valor não era atingido. Então para não termos problema com a coleta de dados foi coletado para todas as frequências até o ângulo de 30°.

Devido ao fato dos dados terem sido coletados em dias diferentes foi necessário reunir esses arquivos coletados, em formato .csv, em um único arquivo, e posteriormente foram retirados os dados e arquivos duplicados. Dessa forma, esse procedimento é descrito nos códigos a seguir:

```
1 import pandas as pd
2 import os
3
4 directory = r'C:\Users\laisa\OneDrive\Area de Trabalho\AI'
5
6 excel_files = [
7     'DadosAutomacaoInteligente30hz0a30g.xlsx',
8     'DadosAutomacaoInteligente35hz0a30g.xlsx',
9     'DadosAutomacaoInteligente40hz30a0g.xlsx',
10    'DadosAutomacaoInteligente45hz0a30g.xlsx',
```

```

11     'DadosAutomacaoInteligente50hz0a30g.xlsx'
12 ]
13
14 dfs = []
15
16 for file in excel_files:
17     file_path = os.path.join(directory, file)
18     df = pd.read_excel(file_path)
19     dfs.append(df)
20
21 combined_df = pd.concat(dfs, ignore_index=True)
22
23 output_file = os.path.join(directory, 'CombinedData.xlsx')
24 combined_df.to_excel(output_file, index=False)
25
26 print(f"Combined data saved to {output_file}")

```

Listing 1: Coleta de dados.

Após a coleta dos dados, é essencial realizar a limpeza e normalização para garantir a qualidade e eficiência do conjunto de dados. Esse processo minimiza a ocorrência de erros durante o treinamento do modelo, assegurando que os dados estejam consistentes, livres de ruídos e prontos para análise.

```

1
2 import pandas as pd
3 from sklearn.preprocessing import MinMaxScaler
4
5 file_path = r'C:\Users\laisa\OneDrive\Area de Trabalho\AI\CombinedData.
6     .xlsx'
7 xls = pd.ExcelFile(file_path)
8
9 df = pd.read_excel(file_path, sheet_name='Sheet1')
10
11 df.columns = [
12     "Timestamp",
13     "Inversor",
14     "CV_4A_READ",
15     "PT_1A",
16     "PT_1B",
17     "PT_1C",
18     "PT_1D",
19     "PT_2A",
20     "PT_2B",
21     "PT_2C",
22     "PT_3A",
23     "PT_3B",
24     "PT_4A",
25     "PT_5A",
26     "PT_6A",
27     "FT_1A",
28     "FT_2A",
29     "FT_2B",
30     "FT_3A",

```

```

31     "FT_4A",
32     "FT_4B",
33     "FT_5A",
34     "FT_6A"
35 ]
36
37 df = df.drop_duplicates()
38
39 for column in df.columns:
40     if column != 'Timestamp':
41         df[column] = df[column].astype(float)
42
43 #scaler = MinMaxScaler()
44 #df[df.columns.difference(['Timestamp'])] = scaler.fit_transform(df[df.
45     columns.difference(['Timestamp'])])
46
47 df_info = df.info()
48
49 print(df.head())
50 print(df_info)
51
52 output_file_path = r'C:\Users\laisa\OneDrive\Area de Trabalho\AI\
53     DadosTratados.xlsx'
54 df.to_excel(output_file_path, index=False)
55
56 print(f"Dados normalizados salvos em: {output_file_path}")

```

Listing 2: Normalização dos dados.

4 Modelagem da Planta

A partir da normalização de dados adquiridos experimentalmente no laboratório, pode-se realizar a análise da correlação entre eles. Entende-se que o sistema funciona com um conjunto de variáveis, as quais são formadas por fatores como pressão, temperatura e frequência. Com isso, a partir dessa combinação de valores de entrada, pode-se realizar a predição de valor de saída da variável PT_A4. As diversas variáveis influenciam o valor de saída com pesos diferentes, assim para compreender melhor a correlação entre as variáveis de entrada do sistema foi desenvolvido o gráfico ilustrado na Figura 2.

```

1
2 import pandas as pd
3 import numpy as np
4 import seaborn as sns
5 import matplotlib.pyplot as plt
6
7 file_path = 'DadosTratados.xlsx'
8 data = pd.read_excel(file_path)
9
10 correlation_matrix = data.corr()
11 pt_4a_correlation = correlation_matrix["PT_4A"].drop("PT_4A")
12
13 plt.figure(figsize=(10, 8))

```

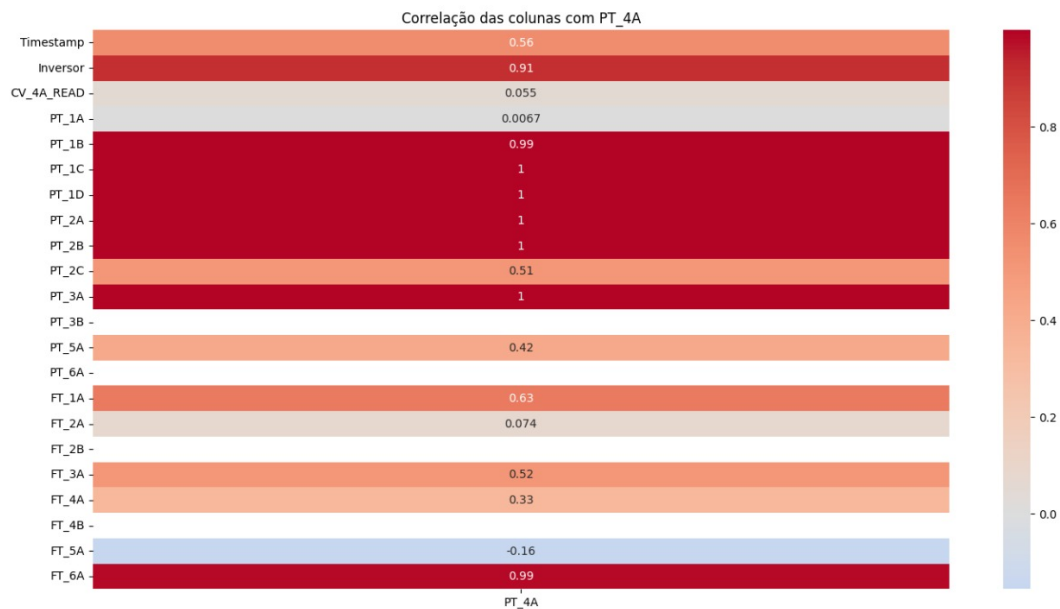
```

14 sns.heatmap(pt_4a_correlation.to_frame(), annot=True, cmap='coolwarm',
    center=0)
15 plt.title("Correlacao das colunas com PT_4A")
16 plt.show()

```

Listing 3: Correlação entre as variáveis.

Figura 2: Correlação dos dados obtidos experimentalmente.



5 Desenvolvimento da Rede Neural Artificial

Após essa análise foi desenvolvido o modelo de treinamento da Rede Neural, no qual foi considerada a condição de um MAPE menor do que 5% para treinamento da rede e retiradas as variáveis que tinham correlação nula com a variável principal PT_04, de acordo com a Figura 2.

O modelo de previsão inicial do sistema foi desenvolvido em Python e utilizado o seguinte código:

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import Dense
6 from tensorflow.keras.optimizers import Adam
7 from tensorflow.keras.callbacks import Callback, EarlyStopping
8 import matplotlib.pyplot as plt
9 import numpy as np

```

```

10
11 file_path = r'C:\Users\laisa\OneDrive\Area de Trabalho\AI\DadosTratados.
    xlsx'
12 data = pd.read_excel(file_path)
13
14 X = data.drop(columns=['Timestamp', 'PT_4A', 'PT_3B', 'PT_6A', 'FT_2B', '
    FT_4B', 'FT_5A', 'CV_4A_READ', 'PT_1A', 'FT_2A'])
15 y = data['PT_4A']
16
17 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.7,
    random_state=42)
18
19 scaler = StandardScaler()
20 X_train = scaler.fit_transform(X_train)
21 X_test = scaler.transform(X_test)
22
23 class MAPEStopper(Callback):
24     def on_epoch_end(self, epoch, logs={}):
25         y_pred = self.model.predict(X_test)
26         non_zero_indices = y_test != 0
27         y_test_non_zero = y_test[non_zero_indices]
28         mape = np.mean(np.abs((y_test - y_pred.flatten()) /
    y_test_non_zero)) * 100
29         if mape < 5:
30             self.model.stop_training = True
31             print(f"\nEpoch {epoch+1}: MAPE abaixo de 5% - parando
    treinamento. MAPE: {mape:.2f}%")
32
33 model = Sequential()
34 model.add(Dense(64, input_dim=X_train.shape[1], activation='relu'))
35 model.add(Dense(32, activation='relu'))
36 model.add(Dense(1))
37
38 model.compile(optimizer=Adam(learning_rate=0.001), loss='
    mean_squared_error')
39
40 early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.0005,
    patience=10, verbose=1, mode='min', restore_best_weights=True)
41 mape_stopper = MAPEStopper()
42
43 history = model.fit(X_train, y_train, epochs=250, validation_split=0.2,
    batch_size=32, verbose=1, callbacks=[mape_stopper])
44
45 loss = model.evaluate(X_test, y_test)
46 print(f'MSE: {loss}')
47
48 y_pred = model.predict(X_test)
49 non_zero_indices = y_test != 0
50 y_test_non_zero = y_test[non_zero_indices]
51 mape = np.mean(np.abs((y_test - y_pred.flatten()) / y_test_non_zero)) *
    100
52 print(f'MAPE: {mape}')
53
54 plt.plot(history.history['loss'], label='Treinamento')

```



```

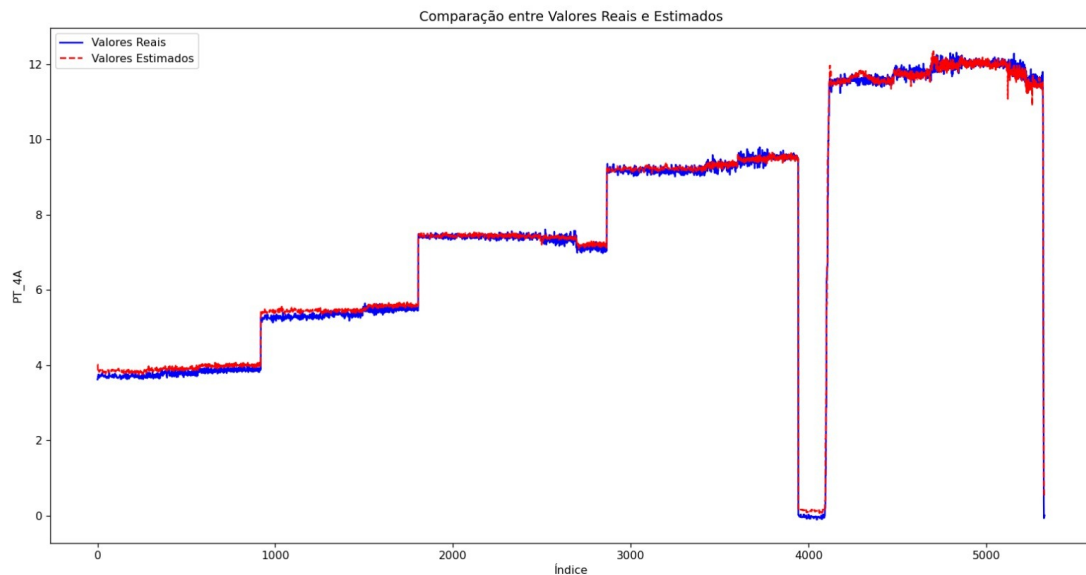
55 plt.plot(history.history['val_loss'], label='Valida o')
56 plt.xlabel('Epochs')
57 plt.ylabel('MSE')
58 plt.title('Treinamento e Valida o')
59 plt.legend()
60 plt.show()
61
62 model.save('Modelo_AI_v1.h5')

```

Listing 4: Modelo RNA.

O modelo de previsão apresenta o seguinte comportamento, ilustrado na Figura 3. Assim, esse foi a base de todo o sistema de controle e previsão desenvolvido no trabalho.

Figura 3: Modelo de previsão desenvolvido.



6 Ajuste no modelo de previsão

Inicialmente, o projeto deveria contar com a implementação do sistema na planta, porém diante das dificuldades enfrentadas pelo grupo, o professor optou por realizar essa atividade somente em simulação.

Com isso, o sistema foi modelado em Python seguindo os passos:

6.1 Criação da RNA

Assim como o modelo criado inicialmente, objetiva-se o desenvolvimento de uma Rede Neural Artificial, que realize o modelo de previsão da variável PT_04, com um MAPE menor que 5%. Para esse ponto foi feito o código a seguir.

```

1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.preprocessing import StandardScaler

```

```

4 from tensorflow.keras.models import Model
5 from tensorflow.keras.layers import Input, Dense
6 from tensorflow.keras.optimizers import Adam
7 from tensorflow.keras.callbacks import Callback, EarlyStopping
8 import matplotlib.pyplot as plt
9 import numpy as np
10
11 file_path = r'C:\Users\laisa\Downloads\Malha Fechada\Malha Fechada\
    DadosTratados.xlsx'
12 data = pd.read_excel(file_path)
13
14 X = data.drop(columns=['Timestamp', 'PT_4A', 'PT_3B', 'PT_6A', 'FT_2B', '
    FT_4B', 'FT_5A', 'CV_4A_READ', 'PT_1A', 'FT_2A'])
15 y = data['PT_4A']
16
17
18 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.7,
    random_state=42)
19
20
21 scaler = StandardScaler()
22 X_train = scaler.fit_transform(X_train)
23 X_test = scaler.transform(X_test)
24
25 # Callback para parar treinamento baseado no MAPE
26 class MAPEStopper(Callback):
27     def on_epoch_end(self, epoch, logs={}):
28         y_pred = self.model.predict(X_test)
29         non_zero_indices = y_test != 0
30         y_test_non_zero = y_test[non_zero_indices]
31         mape = np.mean(np.abs((y_test - y_pred.flatten()) /
    y_test_non_zero)) * 100
32         if mape < 5:
33             self.model.stop_training = True
34             print(f"\nEpoch {epoch+1}: MAPE abaixo de 5% - parando
    treinamento. MAPE: {mape:.2f}%")
35
36
37 inputs = Input(shape=(X_train.shape[1],))
38 x = Dense(64, activation='relu')(inputs)
39 x = Dense(32, activation='relu')(x)
40 outputs = Dense(1)(x)
41
42 model = Model(inputs=inputs, outputs=outputs)
43
44
45 model.compile(optimizer=Adam(learning_rate=0.001), loss='
    mean_squared_error')
46
47
48 early_stopping = EarlyStopping(monitor='val_loss', min_delta=0.0005,
    patience=10, verbose=1, mode='min', restore_best_weights=True)
49 mape_stopper = MAPEStopper()
50

```

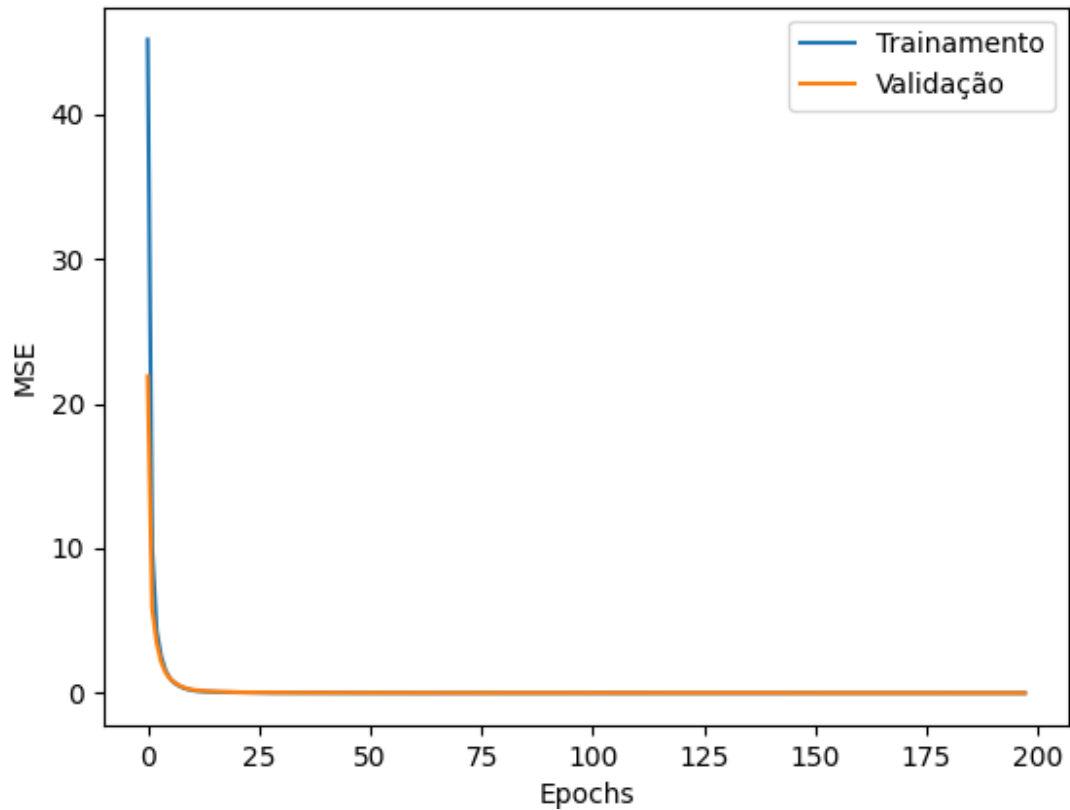
```

51 # Treinamento do modelo
52 history = model.fit(X_train, y_train, epochs=350, validation_split=0.2,
    batch_size=32, verbose=1, callbacks=[mape_stopper])
53
54 # Avalia o do modelo
55 loss = model.evaluate(X_test, y_test)
56 print(f'MSE: {loss}')
57
58 y_pred = model.predict(X_test)
59 non_zero_indices = y_test != 0
60 y_test_non_zero = y_test[non_zero_indices]
61 mape = np.mean(np.abs((y_test - y_pred.flatten()) / y_test_non_zero)) *
    100
62 print(f'MAPE: {mape}')
63
64
65 plt.plot(history.history['loss'], label='Treinamento')
66 plt.plot(history.history['val_loss'], label='Valida o')
67 plt.xlabel('Epochs')
68 plt.ylabel('MSE')
69 plt.title('Treinamento e Validar o')
70 plt.legend()
71 plt.show()
72
73 model.save('Modelo_AI_v1.h5')

```

Listing 5: Modelo RNA.

Figura 4: Validação de Dados.



6.2 Validação do Modelo

Após o desenvolvimento do modelo, foi necessário realizar a validação e etapa de testes para verificar a eficiência. Assim, para essa etapa foram utilizados os códigos a seguir.

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.preprocessing import StandardScaler
4 from tensorflow.keras.models import load_model
5
6
7 model = load_model('Modelo_AI_v1.h5')
8
9
10 new_input = np.array([30, 5.377099, 3.30916, 3.199237, 4.277863, 3.606107,
11                        0.064122, 3.471756, 1.206405, 2.47063, 0.365964,
12                        2.220835, 0.352547])
13
14
15 file_path = r'C:\Users\laisa\Downloads\Malha Fechada\Malha Fechada\
16             DadosTratados.xlsx'
17 data = pd.read_excel(file_path)
18
```

```

19 X = data.drop(columns=['Timestamp', 'PT_4A', 'PT_3B', 'PT_6A', 'FT_2B', '
    FT_4B', 'FT_5A', 'CV_4A_READ', 'PT_1A', 'FT_2A'])
20
21
22 scaler = StandardScaler()
23 X_scaled = scaler.fit_transform(X)
24
25
26 new_input_scaled = scaler.transform([new_input])
27
28
29 prediction = model.predict(new_input_scaled)
30
31
32 print(f'Previs o para o novo input: {prediction[0][0]}')

```

Listing 6: Modelo De Previsão.

```

1
2 import numpy as np
3 import pandas as pd
4 from sklearn.preprocessing import StandardScaler
5 from tensorflow.keras.models import load_model
6 import matplotlib.pyplot as plt
7
8
9 file_path = r'DadosTratados.xlsx'
10 data = pd.read_excel(file_path)
11
12
13 X = data.drop(columns=['Timestamp', 'PT_4A', 'PT_3B', 'PT_6A', 'FT_2B', '
    FT_4B', 'FT_5A', 'CV_4A_READ', 'PT_1A', 'FT_2A'])
14 y = data['PT_4A']
15
16 # Dividir os dados em conjunto de treino e teste
17 from sklearn.model_selection import train_test_split
18 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.7,
    random_state=42)
19
20
21 scaler = StandardScaler()
22 X_train = scaler.fit_transform(X_train)
23 X_test = scaler.transform(X_test)
24
25
26 model = load_model('Modelo_AI_v1.h5')
27
28
29 y_pred = model.predict(X_test).flatten()
30
31 # Ordenar os valores reais e as previs es com base nos valores reais
32 sorted_indices = np.argsort(y_test.values)
33 y_test_sorted = y_test.values[sorted_indices]
34 y_pred_sorted = y_pred[sorted_indices]
35

```

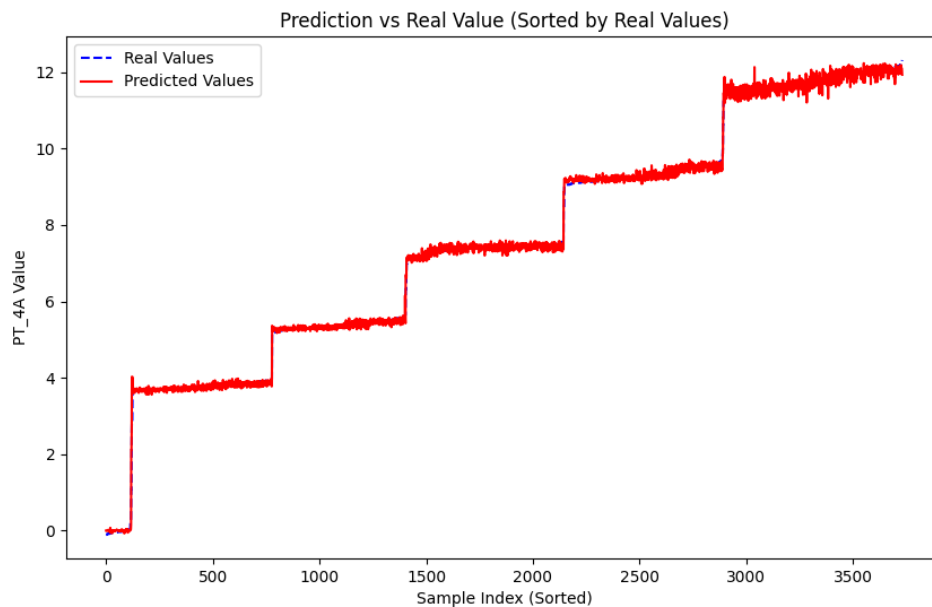
```

36 plt.figure(figsize=(10,6))
37 plt.plot(y_test_sorted, label='Real Values', color='blue', linestyle='--')
38 plt.plot(y_pred_sorted, label='Predicted Values', color='red')
39 plt.title('Prediction vs Real Value (Sorted by Real Values)')
40 plt.xlabel('Sample Index (Sorted)')
41 plt.ylabel('PT_4A Value')
42 plt.legend()
43 plt.show()
44

```

Listing 7: Construção dos Gráficos.

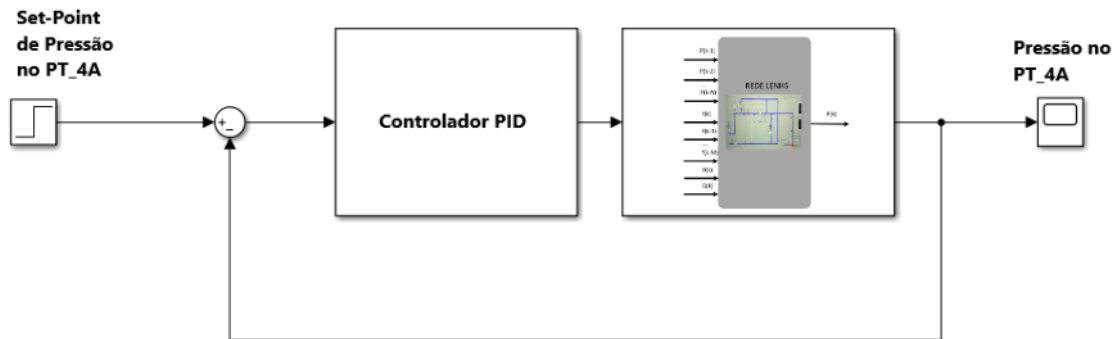
Figura 5: Validação de Dados.



7 Desenvolvimento do sistema de controle PID

Com o modelo da planta elaborada em rede neural, foi elaborado um controlador PID visando efetuar o controle da pressão em PT-4A. O diagrama de blocos referente a situação está apresentado na Figura 6. Nela pode-se observar o *setpoint*, o qual é subtraído do valor real para gerar o valor do erro que passará ao bloco do controlador. Sua saída (a frequência) passará pelo bloco da rede neural, a qual representa a planta.

Figura 6: Diagrama de blocos controlador PID.



O modelo foi implementado em linguagem Python, no qual os parâmetros foram obtidos pelo método da tentativa e erro.

O código pode ser observado abaixo:

```

1
2 #As bibliotecas numpy, pandas, StandardScaler, load_model e matplotlib
  s o importados para manipula o de dados, normaliza o ,
  carregamento de modelos de Machine Learning (RNA), e plotagem de
  gráficos, respectivamente. warnings.filterwarnings("ignore") suprime
  avisos desnecessários.
3 import numpy as np
4 import pandas as pd
5 from sklearn.preprocessing import StandardScaler
6 from tensorflow.keras.models import load_model
7 import matplotlib.pyplot as plt
8 import warnings
9 warnings.filterwarnings("ignore")
10
11 #A classe PID define um controlador PID, que ajusta a variável de
  controle com base no erro entre a medida atual e o valor desejado (
  setpoint). O construtor inicializa os ganhos e outras variáveis.
12 class PID:
13     def __init__(self, Kp, Ki, Kd, setpoint):
14         """
15         Inicializa o controlador PID com os parâmetros fornecidos.
16
17         :param Kp: Ganho Proporcional
18         :param Ki: Ganho Integral
19         :param Kd: Ganho Derivativo
20         :param setpoint: Valor desejado para a saída
21         """
22         self.Kp = Kp
23         self.Ki = Ki
24         self.Kd = Kd
25         self.setpoint = setpoint
26         self.erro_anterior = 0
27         self.integral = 0
28

```

```

29 #0 m todo calcular      respons vel por calcular a a      o de controle
    aplicando os termos P, I e D do controlador PID.
30 def calcular(self, medida, dt):
31     """
32     Calcula a a      o de controle com base na medida atual.
33
34     :param medida: Valor atual da sa da medida
35     :param dt: Intervalo de tempo
36     :return: A      o de controle
37     """
38     erro = self.setpoint - medida
39     self.integral += erro * dt
40     derivada = (erro - self.erro_anterior) / dt if dt > 0 else 0
41     self.erro_anterior = erro
42     return self.Kp * erro + self.Ki * self.integral + self.Kd *
        derivada
43
44 #Carrega um modelo de Rede Neural Artificial (RNA) salvo no arquivo '
    Modelo_AI_v1.h5', o arquivo de dados em formato Excel, remove algumas
    colunas indesejadas e verifica se o n mero de colunas restantes      o
    esperado
45 modelo_path = 'Modelo_AI_v1.h5'
46
47 model = load_model(modelo_path)
48 print(f'Modelo carregado com sucesso de {modelo_path}.')
49
50 file_path = r'DadosTratados.xlsx' # Aten      o em colocar o caminho
    correto
51
52 data = pd.read_excel(file_path)
53 print(f'Dados carregados com sucesso de {file_path}.')
54
55 X = data.drop(columns=['Timestamp', 'PT_4A', 'PT_3B', 'PT_6A', 'FT_2B',
56                        'FT_4B', 'FT_5A', 'CV_4A_READ', 'PT_1A', 'FT_2A'])
57
58 if X.shape[1] != 13:
59     print(f'N mero de colunas selecionadas para entrada      {X.shape[1]},
        esperado 13.')
60
61 #Normaliza os dados de entrada utilizando StandardScaler para ajustar a
    escala dos dados com base nas caracter sticas do conjunto de
    treinamento.
62 scaler = StandardScaler()
63 scaler.fit(X)
64
65 #Define um valor de setpoint (5.0) e inicializa o controlador PID com
    ganhos P, I e D.
66 setpoint = 5.0 # Defina aqui o valor desejado para a sa da
67 pid = PID(Kp=1, Ki=0.1, Kd=0.05, setpoint=setpoint)
68 print('Controlador PID inicializado.')
69
70 #Define o intervalo de tempo dt e o n mero de passos num_steps para a
    simula      o, al m de vari veis para armazenar o hist rico de
    setpoints, sa das e controles. entrada_atual      a primeira amostra dos

```



```

    dados de entrada.
71 dt = 1.0
72 num_steps = 1200
73 historico_setpoint = []
74 historico_saida = []
75 historico_controle = []
76 entrada_atual = X.iloc[0].values.copy()
77 print('Entrada inicial definida a partir dos dados de treinamento.')
78
79 #Itera por num_steps, ajusta o setpoint no step 400 e 800 para testar a
    resposta do sistema, escalona a entrada e faz previsões com o modelo
    carregado. A saída de controle é calculada e aplicada à entrada. Há
    um saturador para limitar a variável de controle.
80 for passo in range(num_steps):
81     if passo == 400 :
82         setpoint = 6.0
83         pid = PID(Kp=1, Ki=0.1, Kd=0.05, setpoint=setpoint)
84
85     if passo == 800 :
86         setpoint = 4.0
87         pid = PID(Kp=1, Ki=0.1, Kd=0.05, setpoint=setpoint)
88
89     entrada_scaled = scaler.transform([entrada_atual])
90
91     saida_predita = model.predict(entrada_scaled)[0][0]
92
93     historico_setpoint.append(setpoint)
94     historico_saida.append(saida_predita)
95
96     controle = pid.calcular(saida_predita, dt)
97     historico_controle.append(controle)
98
99     entrada_atual[0] += controle
100
101     entrada_atual[0] = np.clip(entrada_atual[0], 0, 100)
102
103 #Plota os resultados da simulação (setpoint, saída predita e controle)
    ao longo do tempo, para a análise do comportamento do sistema
    controlado.
104 plt.figure(figsize=(12, 6))
105 plt.plot(historico_setpoint, label='Setpoint', linestyle='--')
106 plt.plot(historico_saida, label='Saída Predita', marker='o')
107 plt.xlabel('Passo de Tempo')
108 plt.ylabel('Valor')
109 plt.title('Simulação de Sistema em Malha Fechada')
110 plt.legend()
111 plt.grid(True)
112 plt.tight_layout()
113 plt.show()
114
115 plt.figure(figsize=(12, 4))
116 plt.plot(historico_controle, label='Amplitude de Controle', color='orange')
117 plt.xlabel('Passo de Tempo')
118 plt.ylabel('Controle')

```

```

119 plt.title('A o de Controle ao Longo do Tempo')
120 plt.legend()
121 plt.grid(True)
122 plt.tight_layout()
123 plt.show()

```

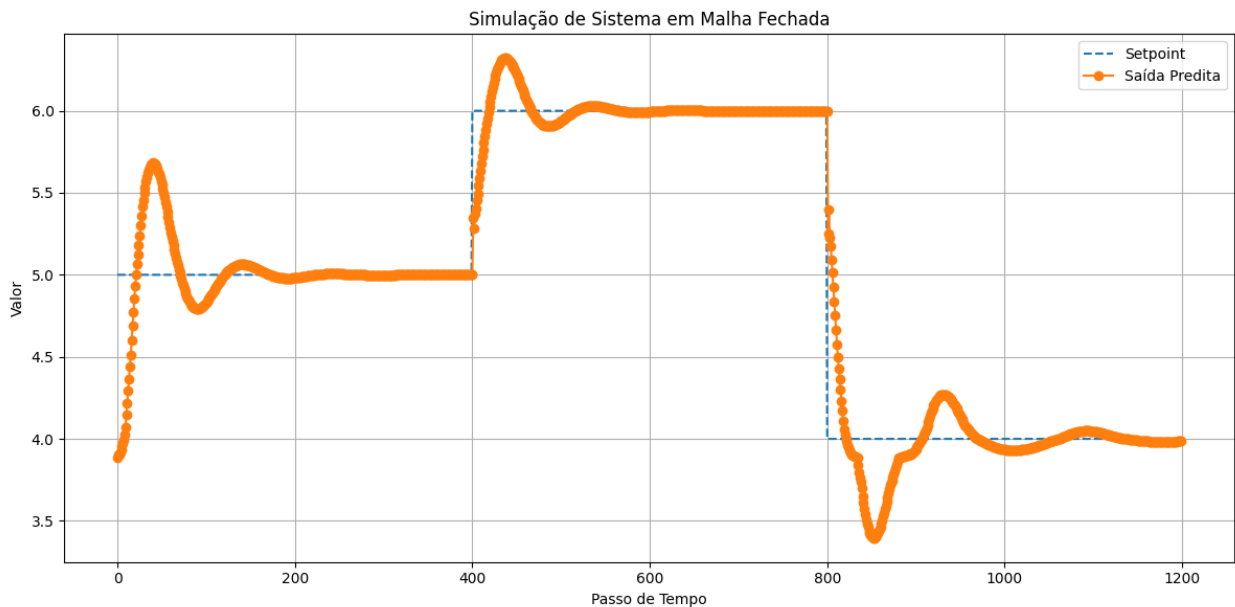
Listing 8: Controlador e Modelo em Malha Fechada.

8 Resultados

Para avaliar o impacto dos diferentes parâmetros do controlador PID no desempenho do sistema, foram realizadas simulações com quatro combinações distintas de ganhos (K_p , K_i , K_d).

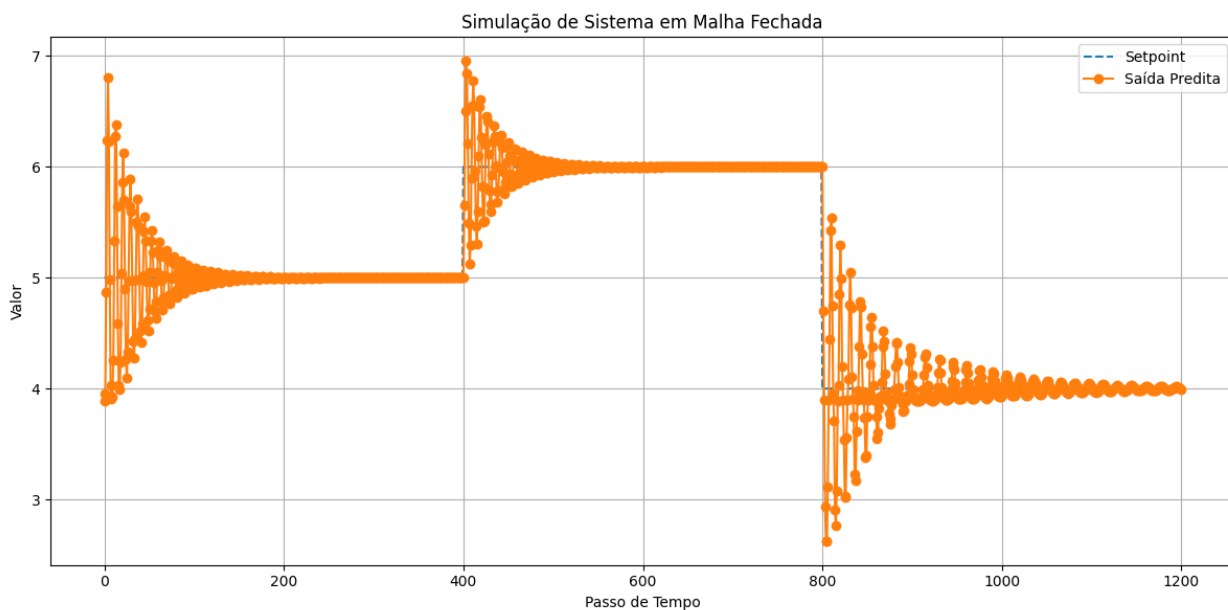
Na Figura 7, temos a resposta do sistema com o controlador configurado com $K_p = 1$, $K_i = 0.1$, $K_d = 5$.

Figura 7: Resposta do Sistema com $K_p = 1$, $K_i = 0.1$, $K_d = 5$.



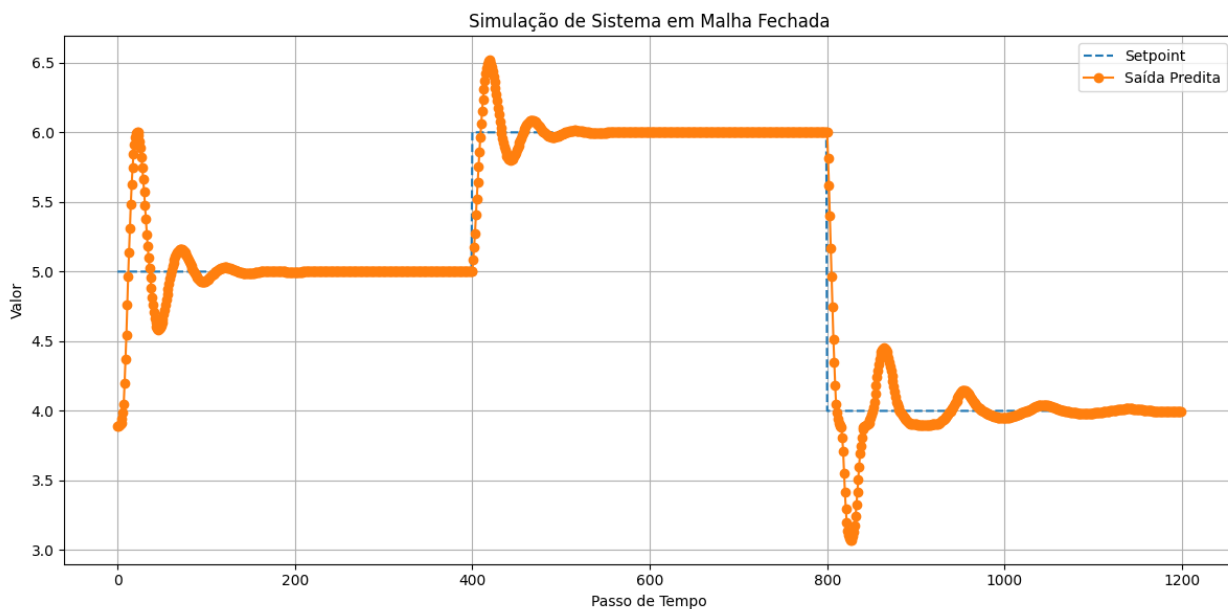
Na Figura 8, a simulação foi realizada com $K_p = 1$, $K_i = 10$, $K_d = 0.05$.

Figura 8: Resposta do Sistema com $K_p = 1$, $K_i = 10$, $K_d = 0.05$.



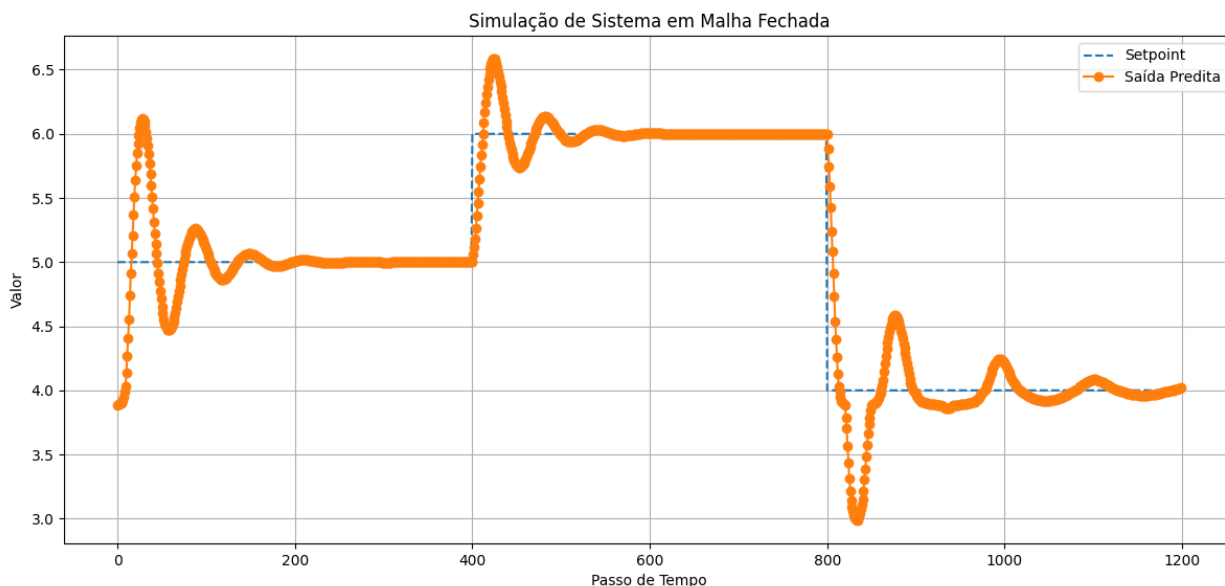
Já na Figura 9, o controlador foi ajustado para $K_p = 1.2$, $K_i = 0.3$, $K_d = 0.05$.

Figura 9: Resposta do Sistema com $K_p = 1.2$, $K_i = 0.3$, $K_d = 0.05$.



Por fim, na Figura 10, temos o controlador com $K_p = 0.8$, $K_i = 0.2$, $K_d = 0.05$.

Figura 10: Resposta do Sistema com $K_p = 0.8$, $K_i = 0.2$, $K_d = 0.05$.



9 Considerações finais

A realização de atividade práticas como essa é fundamental para o aprendizado e verificação da aplicabilidade dos modelos Redes Neurais Artificiais. Com base nos experimentos de simulação realizados e no desenvolvimento da rede neural, foi interessante a tentativa de desenvolver o sistema de previsão e aplica-lo, mesmo diante de circunstancias adversas, em um modelo real. Com isso, apesar de divergências iniciais encontradas, o modelo de previsão e controle implementado respondeu de maneira satisfatória.